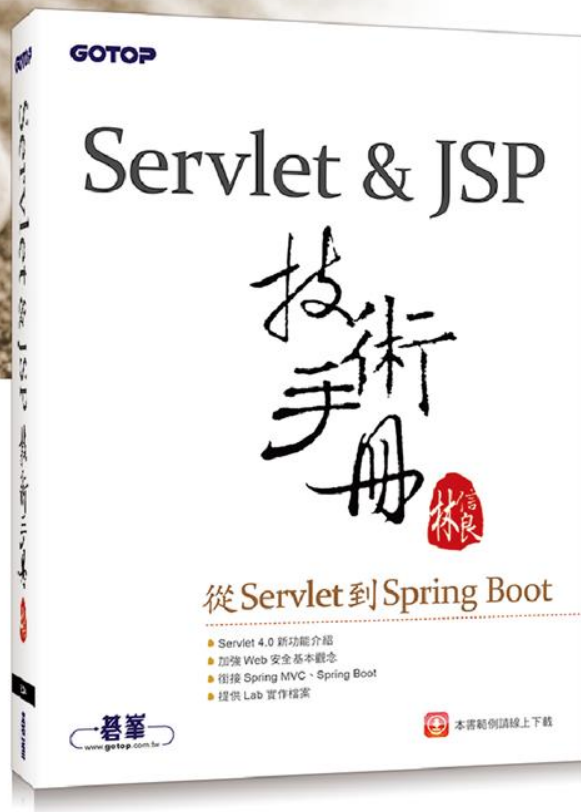




Servlet & JSP

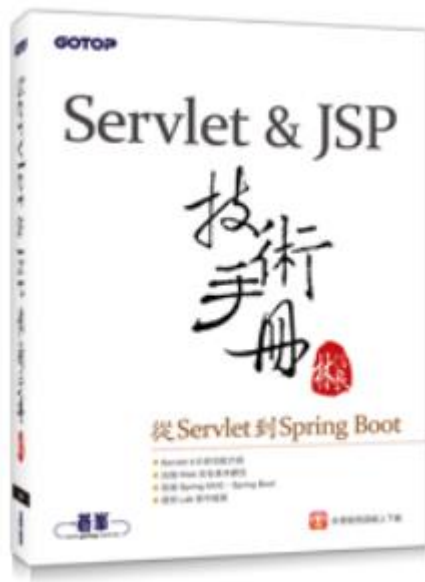
技術手冊

從Servlet到Spring Boot

- Servlet 4.0 新功能介紹
- 加強 Web 安全基本觀念
- 銜接 Spring MVC、Spring Boot
- 提供 Lab 實作檔案

CHAPTER 2

• 撰寫與設定Servlet

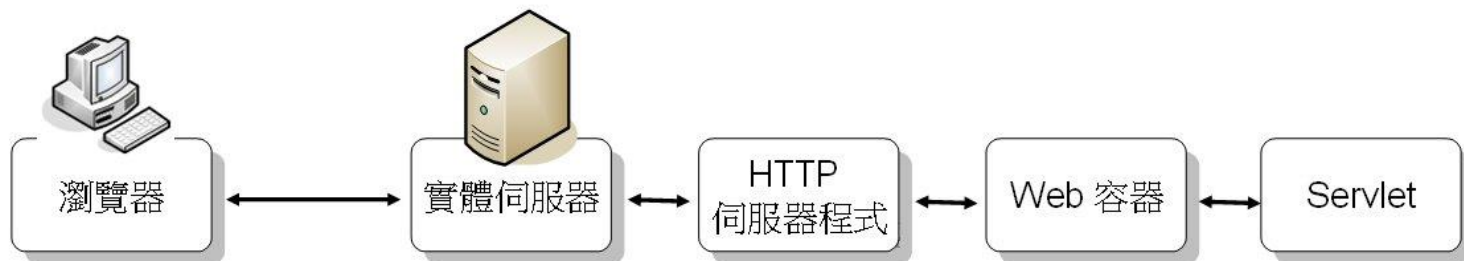


學習目標

- 開發環境準備與使用
- 了解Web應用程式架構
- Servlet撰寫與部署設定
- 了解URI模式對應
- 使用web-fragment.xml

準備開發環境

- Java SE 8
- Eclipse IDE for Java EE Developers
- Tomcat 9



第一個Servlet程式

```
package cc.openhome;

import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/hello")
public class Hello extends HttpServlet { ← ❶ 繼承 HttpServlet
    @Override
    protected void doGet( ← ❷ 重新定義 doGet()
        HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
```

③ 設定回應內容類型

```
response.setContentType("text/html;charset=UTF-8");
```

```
String name = request.getParameter("name");
```

④ 取得請求參數

```
PrintWriter out = response.getWriter();
```

⑤ 取得回應輸出物件

```
out.print("<!DOCTYPE html>");
```

```
out.print("<html>");
```

```
out.print("<head>");
```

```
out.print("<title>Hello</title>");
```

```
out.print("</head>");
```

```
out.print("<body>");
```

```
out.printf("<h1> Hello! %s!\n</h1>", name);
```

⑥ 跟使用者說 Hello!

```
out.print("</body>");
```

```
out.print("</html>");
```

```
}
```



關於HttpServlet

- Servlet API的相關類別

```
import javax.servlet.ServletException;  
import javax.servlet.annotation.WebServlet;  
import javax.servlet.http.HttpServlet;  
import javax.servlet.http.HttpServletRequest;  
import javax.servlet.http.HttpServletResponse;
```

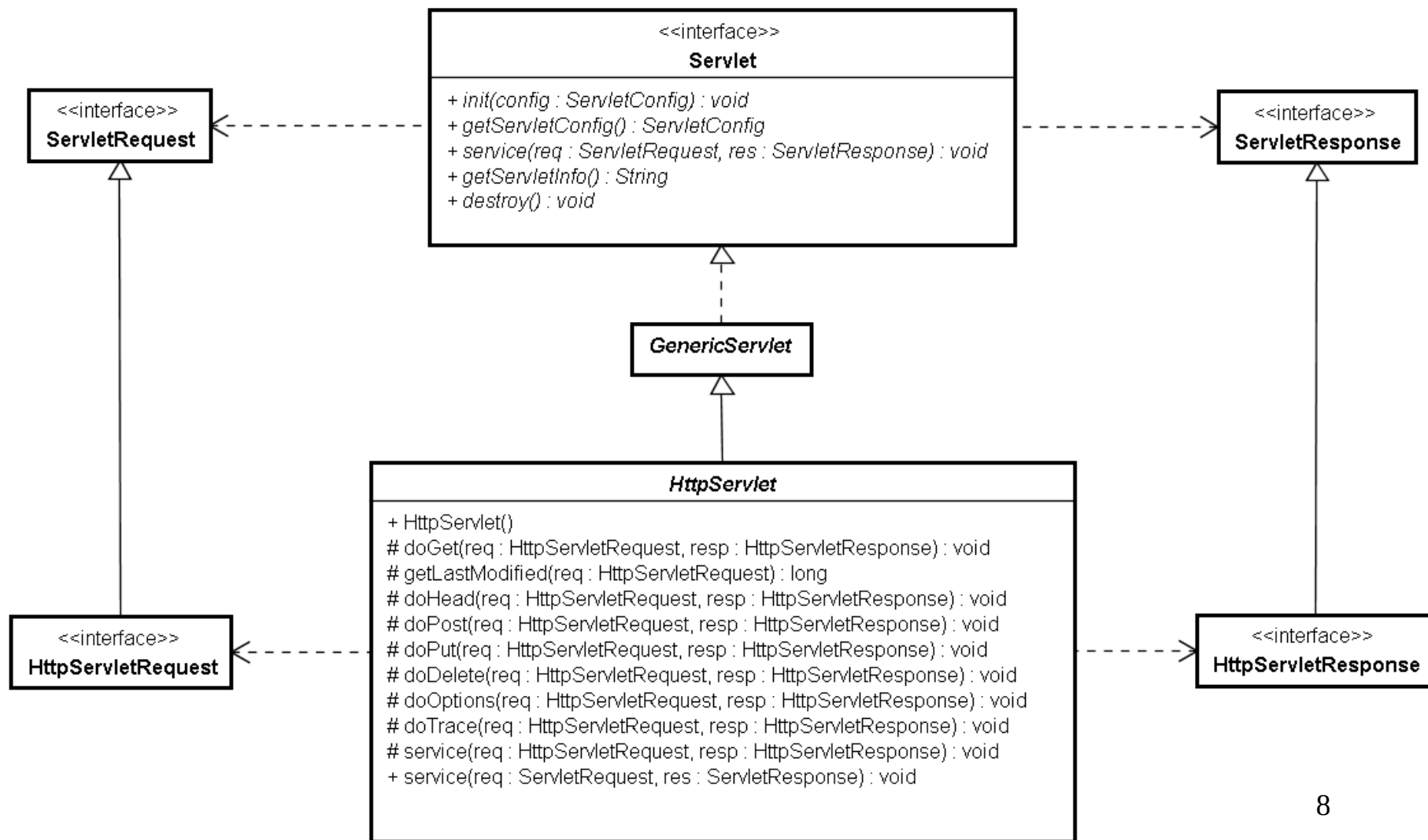
- 類別路徑（Classpath）中必須包括Servlet API的相關類別
 - 手動...

```
% cd YourWorkspace/FirstServlet  
% javac -classpath Yourlibrary/YourTomcat/lib/servlet-api.jar -d ./classes  
src/cc/openhome/Hello.java
```

– IDE...

- ▼ Libraries
 - ▼ Apache Tomcat v9.0 [Apache Tomcat v9.0]
 - > annotations-api.jar - C:\workspace\apache-tomcat-9.0.2\lib
 - > catalina-ant.jar - C:\workspace\apache-tomcat-9.0.2\lib
 - > catalina-ha.jar - C:\workspace\apache-tomcat-9.0.2\lib
 - > catalina-storeconfig.jar - C:\workspace\apache-tomcat-9.0.2\lib
 - > catalina-tribes.jar - C:\workspace\apache-tomcat-9.0.2\lib
 - > catalina.jar - C:\workspace\apache-tomcat-9.0.2\lib
 - > ecj-4.6.3.jar - C:\workspace\apache-tomcat-9.0.2\lib
 - > el-api.jar - C:\workspace\apache-tomcat-9.0.2\lib
 - > jasper-el.jar - C:\workspace\apache-tomcat-9.0.2\lib
 - > jasper.jar - C:\workspace\apache-tomcat-9.0.2\lib
 - > jaspic-api.jar - C:\workspace\apache-tomcat-9.0.2\lib
 - > jsp-api.jar - C:\workspace\apache-tomcat-9.0.2\lib
 - > servlet-api.jar - C:\workspace\apache-tomcat-9.0.2\lib
 - > tomcat-api.jar - C:\workspace\apache-tomcat-9.0.2\lib
 - > tomcat-coyote.jar - C:\workspace\apache-tomcat-9.0.2\lib

關於HttpServlet



關於HttpServlet

```
protected void service(HttpServletRequest req,
                        HttpServletResponse resp)
    throws ServletException, IOException {
    String method = req.getMethod(); // 取得請求的方法
    if (method.equals(METHOD_GET)) { // HTTP GET
        // 略...
        doGet(req, resp);
        // 略 ...
    } else if (method.equals(METHOD_HEAD)) { // HTTP HEAD
        // 略 ...
        doHead(req, resp);
    } else if (method.equals(METHOD_POST)) { // HTTP POST
        // 略 ...
        doPost(req, resp);
    } else if (method.equals(METHOD_PUT)) { // HTTP PUT
        // 略 ...
    }
}
```

使用@WebServlet

- Servlet 3.0可使用標註

```
@WebServlet("/hello")  
public class Hello extends HttpServlet {
```

```
    @WebServlet(  
        name="Hello",  
        urlPatterns={"/hello"},  
        loadOnStartup=1  
    )
```

使用web.xml

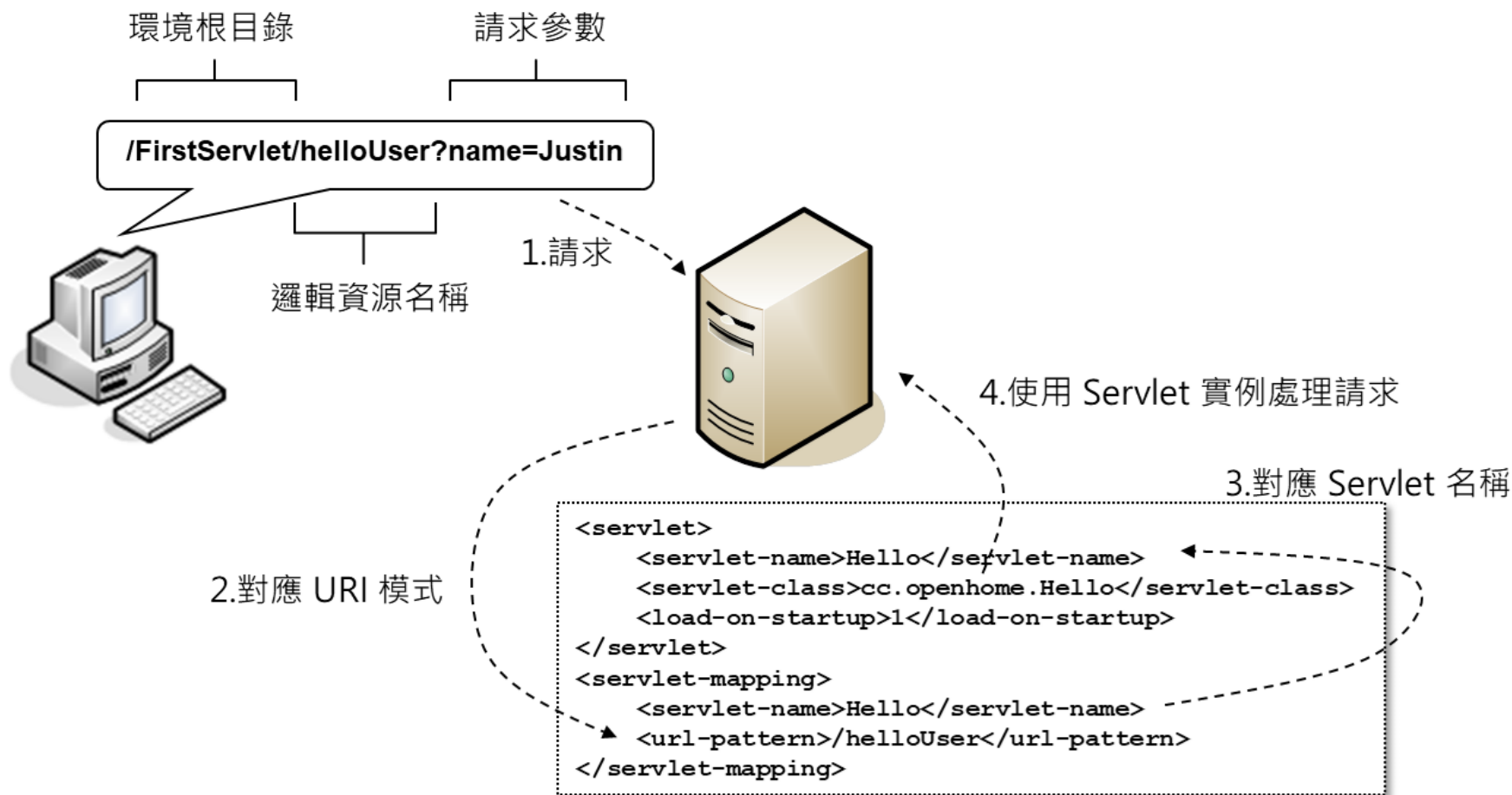
- 部署描述檔（Deployment Descriptor，DD檔）

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd" version="4.0">
    ...略
    <servlet>
        <servlet-name>Hello</servlet-name>
        <servlet-class>cc.openhome.Hello</servlet-class>
        <load-on-startup>1</load-on-startup>
    </servlet>
    <servlet-mapping>
        <servlet-name>Hello</servlet-name>
        <url-pattern>/helloUser</url-pattern>
    </servlet-mapping>
</web-app>
```

使用web.xml

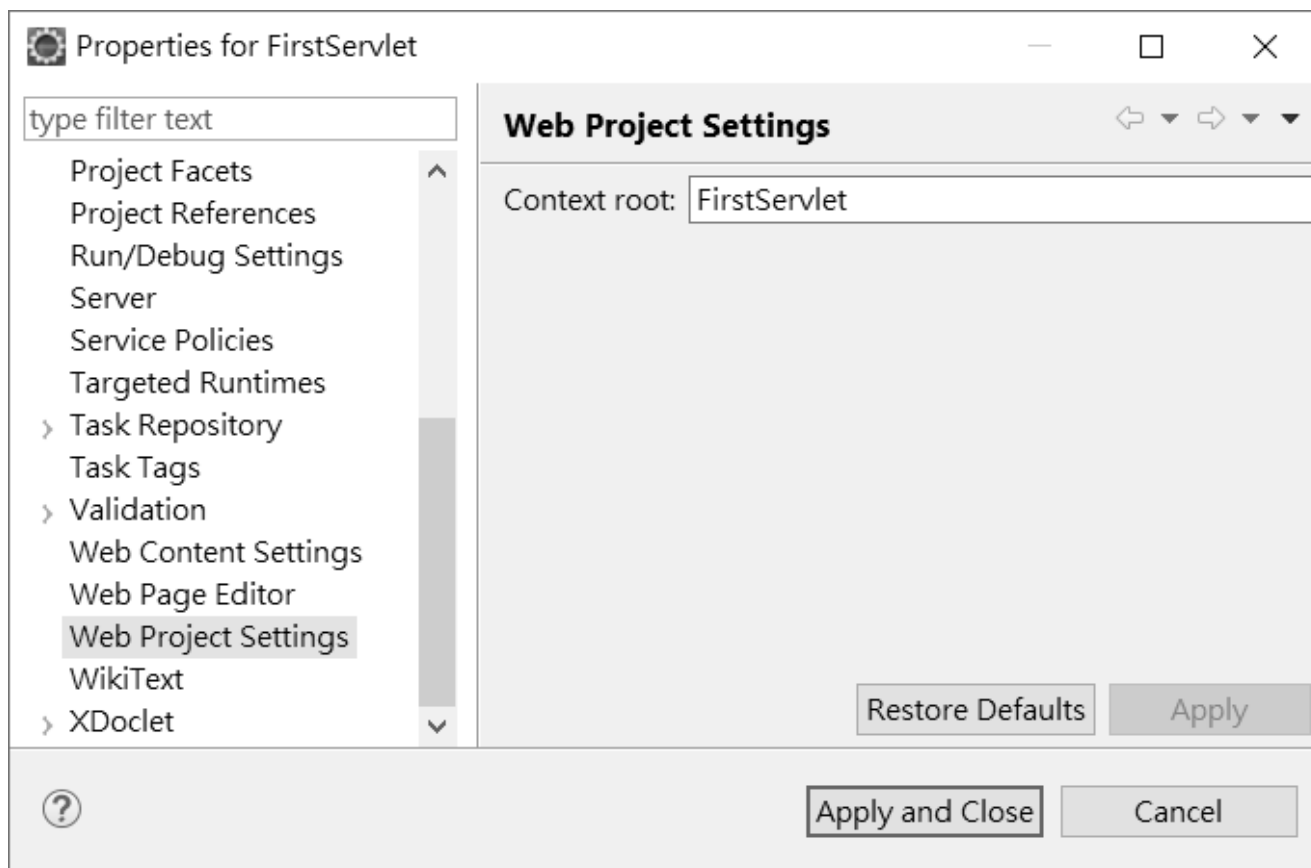
- web.xml中的設定會覆蓋Servlet中的標註設定
 - 可以使用標註來作預設值
 - web.xml來作日後更改設定值之用

使用web.xml



環境根目錄 (Context Root)

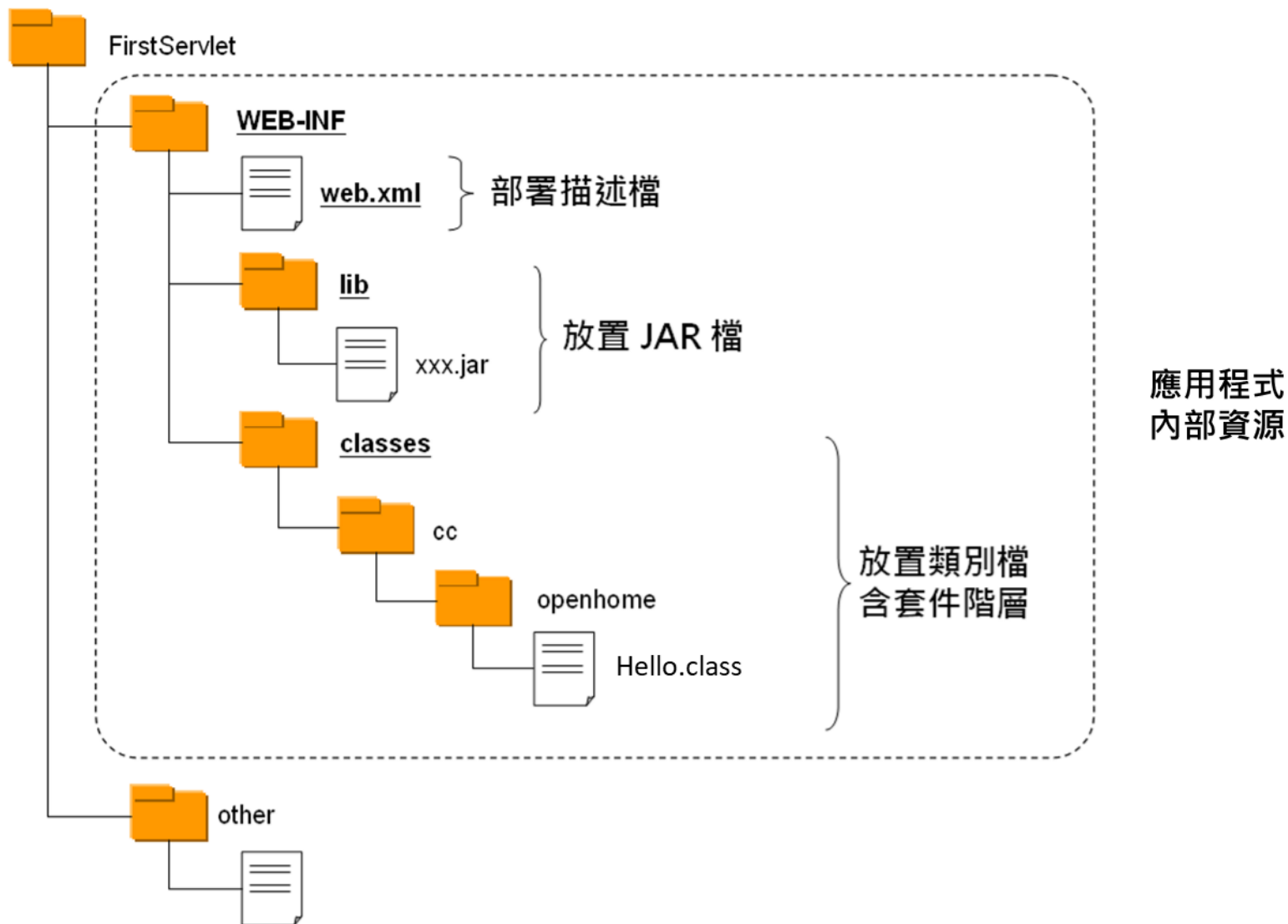
- 設定方式因使用的Web應用程式伺服器而不同



使用web.xml

- 一個Servlet在web.xml中會有三個名稱設定
 - `<url-pattern>`設定的邏輯名稱
 - `<servlet-name>`註冊的Servlet名稱
 - `<servlet-class>`設定的實體類別名稱

檔案組織與部署



檔案組織與部署

- 實際上在部署Web應用程式時，會將Web應用程式封裝為一個WAR（Web Archive）檔案
 - 副檔名為*.war的檔案
 - 可使用JDK所附的jar工具程式來建立
 - 在Eclipse可以執行「Export/WAR file」匯出WAR
- Tomcat容器若發現webapps目錄中有WAR檔案，會將之解壓縮，並載入Web應用程式

URI模式設定

- 請求URI

`requestURI = contextPath + servletPath + pathInfo`

- 環境路徑
- Servlet路徑
- 路徑資訊

- 可使用`HttpServletRequest`的
`getRequestURI ()` 來取得

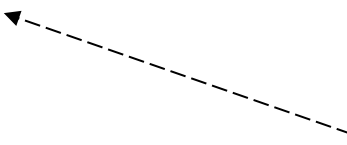
環境路徑

- 容器用來決定該挑選哪個Web應用程式的依據
- 環境路徑的設定方式標準中並沒有規範，依使用的應用程式伺服器而有所不同
- 可使用HttpServletRequest的 **getContextPath()** 來取得
- 如果應用程式環境路徑與Web伺服器環境根路徑相同，則應用程式環境路徑為空字串
- 如果不是，則應用程式環境路徑以"/"開頭，不包括"/"結尾

URI模式設定

- 路徑對應（Path mapping）
 - `"/`開頭但`/*`結尾的URI模式，例如`/guest/*`
- 延伸對應（extension mapping）
 - 以`*.`開頭的URI模式，例如`*.view`
- 環境根目錄（Context root）對應
- 預設Servlet
 - 僅包括`/`的URI模式
- 嚴格匹配（Exact match）
 - 不符合以上設定的其他字串，例如`/guest/test.view`

URI模式設定

- 如果URI模式在設定比對的規則在某些URI請求時有所重疊，則請求時比對的原則是從最嚴格的URI模式開始符合
 - `"/admin/login.do"`
 - `"/admin/*"`
 - `"*.do"`
- 請求 `/admin/login.do`
- 

Servlet路徑

- 不包括路徑資訊（ Path info ）與請求參數（ Request parameter ）
- Servlet路徑直接對應至URI模式資訊
- 可使用HttpServletRequest的 **getServletPath()** 來取得
- 基本上是以"/"開頭
- 在"/*"與""的情況下，getServletPath() 取得的Servlet路徑是空字串

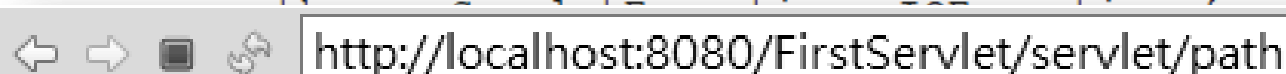
Servlet路徑

- 若請求是根據 `"/hello.do"` 對應至某個 Servlet
 - Servlet路徑就是 `"/hello.do"`
- 如果透過 `"/servlet/*"` 對應至 Servlet
 - Servlet路徑就是 `"/servlet"`
- 如果透過 `"/*" 或 ""` 對應至 Servlet
 - Servlet路徑就是空字串

路徑資訊

- 不包括請求參數，指的是不包括環境路徑與Servlet路徑部份的額外路徑資訊
- 可使用`HttpServletRequest`的 **`getPathInfo()`** 來取得
 - 如果沒有額外路徑資訊，則為`null`


```
@WebServlet("/servlet/*")
public class Path extends HttpServlet {
    protected void doGet(
        HttpServletRequest request, HttpServletResponse response)
```

A screenshot of a web browser window. The address bar shows the URL "http://localhost:8080/FirstServlet/servlet/path". The browser interface includes back, forward, and search buttons.

```
/FirstServlet/servlet/path
/FirstServlet
/servlet
/path
```

```
    out.print("<body>");
    out.printf("%s<br>", request.getRequestURI());
    out.printf("%s<br>", request.getContextPath());
    out.printf("%s<br>", request.getServletPath());
    out.print(request.getPathInfo());
    out.print("</body>");
    out.print("</html>");
```

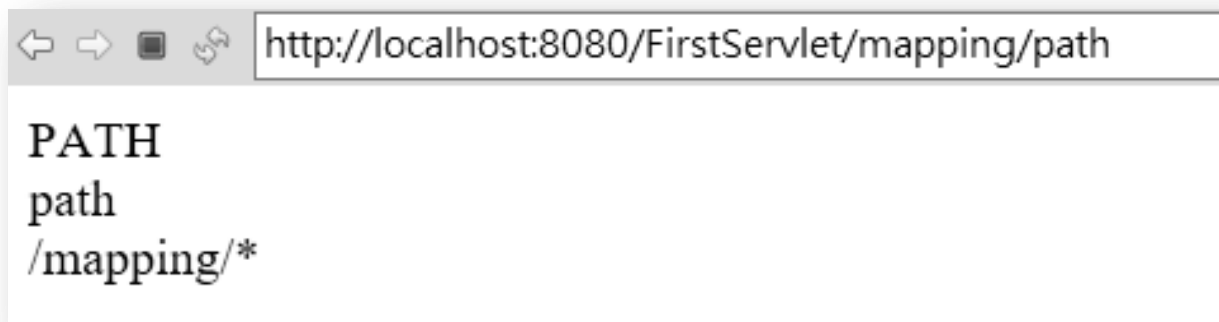
```
}
```

```
}
```

HttpServletMapping

- 在Servlet 4.0中，HttpServletRequest新增了getHttpServletMapping()方法

```
out.printf("%s<br>", mapping.GetMappingMatch());  
out.printf("%s<br>", mapping.getMatchValue());  
out.print(mapping.getPattern());
```



Web目錄結構

- 一個Web應用程式基本上會由以下項目組成：
 - 靜態資源（HTML、圖片、聲音等）
 - Servlet
 - JSP
 - 自定義類別
 - 工具類別
 - 部署描述檔（web.xml等）、設定資訊（Annotation等）

WEB-INF

- 不被列入應用程式根目錄中可直接存取項目
 - /WEB-INF/web.xml 是部署描述檔。
 - /WEB-INF/classes 用來放置應用程式用到的自定義類別（.class），必須包括套件（Package）結構
 - /WEB-INF/lib 用來放置應用程式用到的JAR（Java ARchive）檔案
- 可以透過程式面的控管，讓程式來取得/WEB-INF中的資源，像是使用ServletContext的 `getResource()` 與 `getResourceAsStream()`，或是透過RequestDispatcher請求調派

Web應用程式用到的JAR檔案

- 可以放置Servlet、JSP、自定義類別、工具類別、部署描述檔等
- 應用程式的類別載入器可以從JAR中載入對應的資源
- 可以在JAR檔案的/META-INF/resources目錄中放置靜態資源或JSP等

類別尋找

- /WEB-INF/classes
- /WEB-INF/lib的JAR檔案
- 容器實作本身存放類別或JAR的目錄

歡迎頁面

- web.xml

```
<welcome-file-list>
  <welcome-file>index.html</welcome-file>
  <welcome-file>default.jsp</welcome-file>
</welcome-file-list>
```

- 尋找順序

- 目錄
- JAR的/META-INF/resources

- 如果URI最後是以/結尾，但不存在該目錄，則會使用預設Servlet

使用web-fragment.xml

- Servlet 3.0
 - JAR檔案可用來作為Web應用程式的部份模組
 - 部署描述檔web-fragment.xml
 - 放置在JAR檔案中的META-INF目錄之中

使用web-fragment.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-fragment id="WebFragment_ID" version="4.0"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-fragment_4_0.xsd">
    <display-name> WebFragment1</display-name>
    <name>WebFragment1</name>
    <servlet>
        <servlet-name>Hi</servlet-name>
        <servlet-class>cc.openhome.Hi</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>Hi</servlet-name>
        <url-pattern>/hi</url-pattern>
    </servlet-mapping>
</web-fragment>
```

web.xml與web-fragment.xml

- 決定web.xml與web-fragment.xml的配置順序

```
<web-app ...>
  <absolute-ordering>
    <name>WebFragment1</name>
    <name>WebFragment2</name>
  </absolute-ordering>
  ...
</web-app>
```

- 各個JAR檔中web-fragment.xml定義的名稱不得重複，若有重複，則會忽略掉重複的名稱

另一個定義順序的方式

- 各個JAR中...
- web.xml沒有定義順序資訊
- 載入定義的順序
 - web.xml
 - WebFragment3
 - WebFragment2
 - WebFragment1

```
<web-fragment ...>
  <name>WebFragment1</name>
  <ordering>
    <after><name>MyFragment2</name></after>
  </ordering>
  ...
</web-fragment>

<web-fragment ...>
  <name>WebFragment2</name>
  ..
</web-fragment>

<web-fragment ...>
  <name>WebFragment3</name>
  <ordering>
    <before><others/></before>
  </ordering>
  ..
</web-fragment>
```

metadata-complete屬性

- 設定為true（預設是false），則表示web.xml中已完成Web應用程式的相關定義
 - 部署時不會掃描標註與web-fragment.xml中的定義
 - 如果有<absolute-ordering>與<ordering>也會被忽略

```
<web-app id="WebFragment_ID" version="4.0"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-fragment_4_0.xsd"
    metadata-complete="true">
    ...
</web-app>
```

瑣瑣碎碎

- 如果web-fragment.xml中指定的類別可以在web應用程式的/WEB-INF/classes中找到，就會使用該類別
- 如果該類別本身有標註，而web-fragment.xml又有定義該類別為Servlet，則此時會有兩個Servlet實例
- 如果將<web-fragment>的metadata-complete屬性設定為true（預設是false），就只會處理自己JAR檔案中的標註資訊